



UNIVERSIDAD DE CIENCIAS EMPRESARIALES Y SOCIALES

Carrera: Tecnicatura Universitaria en Programación

Asignatura: Programación II

Nombre del/a docente: Mario Daniel Detke

Nombre del/a estudiante: Gonzalo Ezequiel Beloqui

Nombre de la actividad: Unidad 1 - Actividad 1

Fecha de entrega: 28/03/2026

PROYECTO: Sistema de turnos de centro de salud de cercanía

Fundamentación

El proyecto consiste en desarrollar un sistema de gestión de turnos para un centro de salud local, el cual permitirá registrar pacientes y profesionales, crear agendas de atención, asignar y anular turnos, y enviar recordatorios de consultas. El tema surge del boceto trabajado en la materia Diseño Orientado a Objetos (2024), donde se definieron teóricamente las clases, actividades, secuencias y casos de uso, bajo los parámetros de diseño UML. Link del repositorio público:

<https://github.com/gonezebel/SistemaGestionTurnos.git>

La propuesta es pertinente y factible de prototipar en Java, porque permite aplicar correctamente encapsulamiento, herencia, polimorfismo, interfaces, constructores, getters, setters y métodos de negocio en clases representativas del dominio.

Lineamientos del proyecto

- Nombre del proyecto: beloqui_gonzalo
- Paquete para guardar las clases de los objetos: com.beloqui.modelo
- Paquete para guardar el archivo principal: com.beloqui.main

- Nombres de las clases: representativos del contenido de cada clase
- Nombre del archivo principal: Principal.java

Estructura del proyecto

```
beloqui_gonzalo
|- 00_principal
|  \- Principal.java
|- 01_modelo
|  |- Persona.java
|  |- Paciente.java
|  |- Profesional.java
|  |- Agenda.java
|  |- Turno.java
|  |- Notificacion.java
|  \- Notificable.java
|- 02_controlador
\  \- 03_vista
```

Clases principales del proyecto

1. Clase abstracta Persona

- [ClasePersona](#)

Atributos

- String nombre
- String apellido
- String dni
- String telefono

Constructores

- Persona()
- Persona(String nombre, String apellido, String dni, String telefono)

Métodos getters y setters

- getNombre() / setNombre(String nombre)
- getApellido() / setApellido(String apellido)
- getDni() / setDni(String dni)
- getTelefono() / setTelefono(String telefono)

Métodos necesarios

- String getNombreCompleto()

- boolean validarDni()
- String mostrarDatos()

2. Clase Paciente

- [ClasePaciente](#)

Hereda de Persona e implementa la interfaz Notificable.

Atributos

- int numeroHistoriaClinica
- String obraSocial
- String email

Constructores

- Paciente()
- Paciente(String nombre, String apellido, String dni, String telefono, int numeroHistoriaClinica, String obraSocial, String email)

Métodos getters y setters

- getNumeroHistoriaClinica() / setNumeroHistoriaClinica(int numeroHistoriaClinica)
- getObraSocial() / setObraSocial(String obraSocial)
- getEmail() / setEmail(String email)

Métodos necesarios

- boolean validarEmail()
- String obtenerDestinoNotificacion()
- String enviarNotificacion(String mensaje)
- String mostrarDatos()
- String toString()

3. Clase Profesional

- [ClaseProfesional](#)

Hereda de Persona e implementa la interfaz Notificable.

Atributos

- String matricula
- String especialidad
- String emailInstitucional

Constructores

- Profesional()
- Profesional(String nombre, String apellido, String dni, String telefono, String matricula, String especialidad, String emailInstitucional)

Métodos getters y setters

- getMatricula() / setMatricula(String matricula)
- getEspecialidad() / setEspecialidad(String especialidad)
- getEmailInstitucional() / setEmailInstitucional(String emailInstitucional)

Métodos necesarios

- boolean estaDisponible(String estadoAgenda)
- String obtenerDestinoNotificacion()
- String enviarNotificacion(String mensaje)
- String mostrarDatos()
- String toString()

4. Clase Agenda

- [ClaseAgenda](#)

Atributos

- int idAgenda
- Profesional profesional
- String diaSemana
- String horaInicio
- String horaFin
- String estado

Constructores

- Agenda()
- Agenda(int idAgenda, Profesional profesional, String diaSemana, String horaInicio, String horaFin, String estado)

Métodos getters y setters

- getIdAgenda() / setIdAgenda(int idAgenda)
- getProfesional() / setProfesional(Profesional profesional)
- getDiaSemana() / setDiaSemana(String diaSemana)
- getHoraInicio() / setHoraInicio(String horaInicio)
- getHoraFin() / setHoraFin(String horaFin)
- getEstado() / setEstado(String estado)

Métodos necesarios

- void suspenderAgenda()

- void activarAgenda()
- boolean estaActiva()
- String toString()

5. Clase Turno

- [ClaseTurno](#)

Atributos

- int idTurno
- Paciente paciente
- Profesional profesional
- Agenda agenda
- String fecha
- String hora
- String estado

Constructores

- Turno()
- Turno(int idTurno, Paciente paciente, Profesional profesional, Agenda agenda, String fecha, String hora, String estado)

Métodos getters y setters

- getIdTurno() / setIdTurno(int idTurno)
- getPaciente() / setPaciente(Paciente paciente)
- getProfesional() / setProfesional(Profesional profesional)
- getAgenda() / setAgenda(Agenda agenda)
- getFecha() / setFecha(String fecha)
- getHora() / setHora(String hora)
- getEstado() / setEstado(String estado)

Métodos necesarios

- boolean asignarTurno()
- void anularTurno()
- String mostrarResumenTurno()
- String toString()

6. Clase Notificacion

- [ClaseNotificacion](#)

Atributos

- int idNotificacion
- String tipo

- String mensaje
- String fechaEnvio
- boolean enviada

Constructores

- Notificacion()
- Notificacion(int idNotificacion, String tipo, String mensaje, String fechaEnvio, boolean enviada)

Métodos getters y setters

- getIdNotificacion() / setIdNotificacion(int idNotificacion)
- getTipo() / setTipo(String tipo)
- getMensaje() / setMensaje(String mensaje)
- getFechaEnvio() / setFechaEnvio(String fechaEnvio)
- isEnviada() / setEnviada(boolean enviada)

Métodos necesarios

- String enviarA(Notificable destinatario)
- String mostrarDetalle()
- String toString()

Interfaz implementada

Se implementa la interfaz Notificable en las clases Paciente y Profesional.

- [InterfazNotificable](#)

Métodos de la interfaz

- String obtenerDestinoNotificacion()
- String enviarNotificacion(String mensaje)

La interfaz es utilizada por la clase Notificacion, que puede enviar mensajes a cualquier objeto que implemente Notificable.

Métodos sobrescritos

En el proyecto se sobrescriben varios métodos heredados:

- mostrarDatos() en Paciente
- mostrarDatos() en Profesional
- toString() en Paciente
- toString() en Profesional